PROJECT REPORT

AI Sales Analytics Assistant

A natural-language business intelligence application for sales data analysis

Application	AI Sales Analytics Assistant
Version	1.0.0
Backend	Python, FastAPI, SQLite
AI service	Google Gemini API
Frontend	HTML, CSS, JavaScript

Prepared as technical project documentation

1. Project Overview

The **AI Sales Analytics Assistant** is a web-based business intelligence application that allows users to ask questions about sales data in natural language. Instead of manually filtering spreadsheets or writing SQL queries, a user can ask questions such as “What is the total revenue?” or “Which region performs best?” and receive data-backed results.

The application uses a hybrid approach. Python rule-based intent detection handles common question patterns. If a question is not recognized, the Google Gemini API interprets it and returns a structured JSON intent. The backend validates that response and runs the corresponding SQL analytics against an SQLite database. Calculations are performed by SQL/Python rather than by the language model.

2. Problem Statement

Sales information is often stored in tabular files. Extracting useful insights can require spreadsheet knowledge, manual calculations, or familiarity with SQL. This creates friction for users who need quick answers about revenue, products, regions, and monthly performance. The project addresses this by providing a conversational interface over a structured sales database while keeping the numerical analysis deterministic and verifiable.

3. Objectives

- Provide a simple interface for asking sales-related questions in everyday language.
- Identify the intent of each question using rule-based matching and Gemini as a fallback.
- Calculate business metrics using SQL queries against SQLite.
- Validate AI-generated structured output before executing analytics.
- Handle invalid requests and AI service failures with understandable error messages.
- Test the core analytics functions using automated unit tests.

4. Dataset and Data Preparation

The application uses a sales CSV dataset containing **51 transaction records** and fields for transaction ID, order date, product, category, region, sales channel, quantity, unit price, discount amount, and transaction status. The records are loaded into an SQLite table for querying.

Revenue is calculated using the following formula:

Revenue = (Quantity × Unit Price) – **Discount Amount**

For analytical calculations, the application includes only completed transactions with a positive quantity and a valid positive unit price. Returned transactions and records with invalid or missing calculation values are excluded from the calculations, while the original records remain in the dataset.

5. Technologies Used

Area	Technology	Purpose
Programming	Python 3.10	Application logic and analytics
Backend/API	FastAPI, Uvicorn	HTTP endpoints and serving the application
Database	SQLite, SQL	Structured storage and deterministic queries
AI	Google Gemini API	Natural-language intent interpretation
Frontend	HTML, CSS, JavaScript	Chat interface and result display

Area	Technology	Purpose
Validation	Pydantic, Python validation	Request and AI-response checks
Testing	Pytest, HTTPX	Automated and API testing

6. System Architecture

The application follows a request-processing pipeline. Each user question passes through intent detection before it reaches the analytics layer.

1. User Enters a question in the web chat
2. FastAPI Receives and validates the request
3. Python intent detection Checks known question patterns
4. Gemini fallback (when needed) Returns intent and parameters as JSON
5. Validation Checks supported intent and parameters
6. Analytics + SQLite Executes parameterized SQL and calculates metrics
7. Response Returns structured results to the frontend

Rule-based matching reduces unnecessary API calls for common questions. Gemini is used only when Python cannot recognize the wording. The backend does not allow Gemini to directly execute SQL or modify the database.

7. AI Integration

The Google Gemini API is integrated through the Python Google GenAI SDK. The prompt asks the model to classify the question into one of five supported intents and return JSON containing the intent and any required parameters. For month comparisons, the model is asked to extract the two months in MM-YYYY format; a missing year is interpreted as 2026 for this dataset.

Supported intents:

- **total_revenue** — calculate total qualifying sales revenue.
- **top_product** — identify the product with the highest revenue.
- **monthly_revenue** — return revenue grouped by month.
- **top_region** — identify the highest-revenue region.
- **compare_months** — compare revenue for two specified months.

After receiving Gemini's response, the application parses the JSON and validates that it is a dictionary, that the intent is supported, and that month parameters follow the expected format and range. Invalid output is rejected. If the API is unavailable or returns an error, the application responds with a user-friendly message.

Important design decision: Gemini interprets language; it does not calculate revenue. SQL queries perform the actual calculations using the stored transaction data. This separation improves consistency, testability, and traceability.

8. Database and Analytics Implementation

The SQLite database contains a **sales** table. The analytics module provides separate functions for each supported business question. Queries use a shared eligibility rule: status must be Completed, quantity must be present and greater than zero, and unit price must be present and greater than zero. Revenue is aggregated from quantity, unit price, and discount amount.

For month comparisons, the application uses parameterized SQL placeholders for the month values rather than directly inserting user-provided text into the query. The month comparison returns each month's revenue and the difference (second month minus first month).

9. Example Analytical Results

Metric	Result from the current dataset
Total revenue	■14,39,200
Highest-revenue product	Monitor — ■4,73,000
Highest-revenue region	South — ■4,11,200
January 2026 revenue	■2,10,250
February 2026 revenue	■2,34,600
February vs January	■24,350 higher in February

10. Application Screenshots

The following screenshot was supplied from the running development environment. It shows the FastAPI Swagger interface, the available API endpoints, and the backend server running locally.

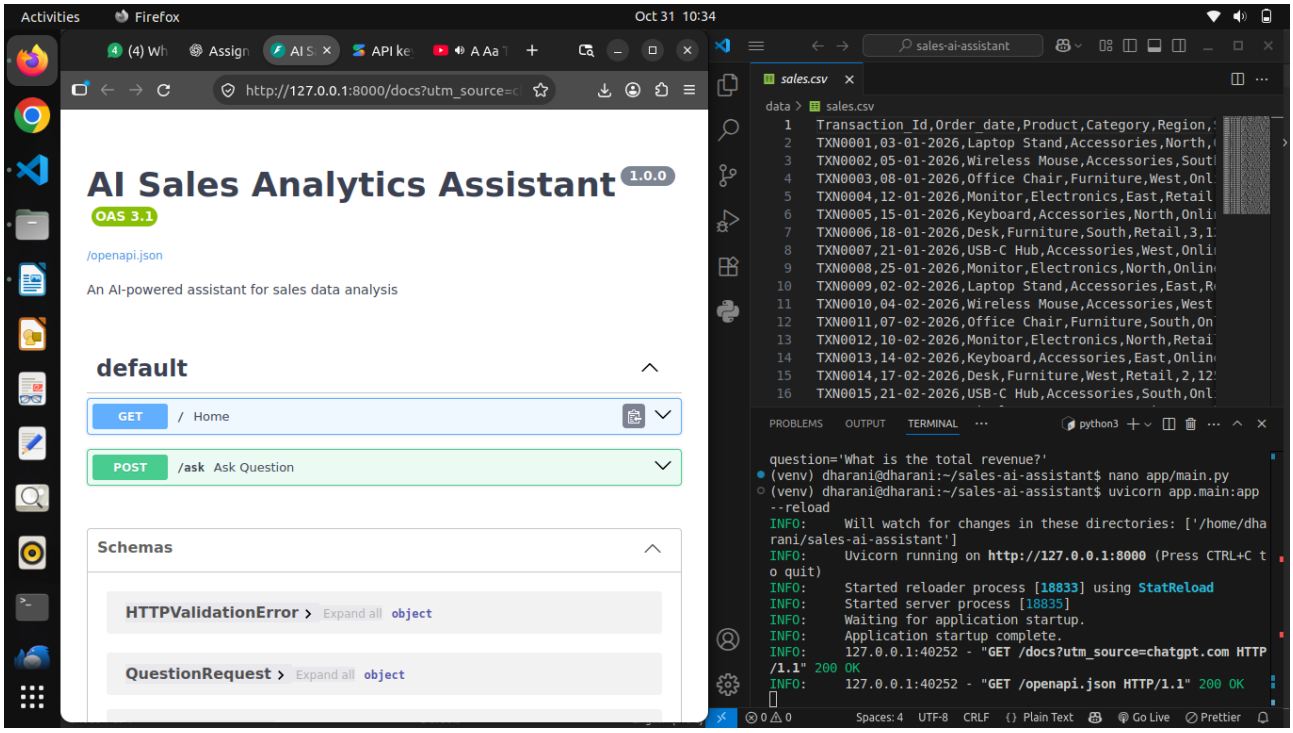


Figure 1. FastAPI Swagger documentation and running local backend.

Note: This report includes the available backend/API screenshot. A separate screenshot of the interactive chat page can be added if a frontend screenshot is captured before final submission.

11. Testing and Validation

The project includes automated tests for the five analytics functions using Pytest. The test suite was run successfully with **5 tests passed**. The tests check the expected total revenue, top product, monthly revenue output, top region, and month-comparison values.

Additional manual checks were performed during development:

- Verified rule-based detection for the five supported intents.
- Tested alternative question wording through the Gemini fallback.
- Checked that unsupported AI intents and invalid month parameters are rejected.
- Simulated Gemini API failure and verified that a friendly error is returned.
- Tested the FastAPI question endpoint and confirmed successful responses.
- Tested the frontend with supported questions and confirmed that results are displayed.

12. Conclusion

The AI Sales Analytics Assistant demonstrates how natural-language processing can make structured business data easier to query. Its hybrid design combines rule-based detection, Gemini-based interpretation, validation, and SQL-backed calculations. The result is a small but practical analytics application in which numerical answers are generated from the database rather than relying on free-form language-model calculations.

13. Future Enhancements

- Add more analytical intents such as sales by category, channel, and product trends.
- Support a wider range of date periods and flexible date filters.
- Add charts and downloadable reports for visual analysis.
- Improve handling of ambiguous questions and ask users for clarification when needed.
- Add authentication, role-based access, and deployment configuration.
- Expand the dataset and introduce additional data-quality checks.

14. Project Structure

```
sales-ai-assistant/
■■■ app/ (FastAPI routes, database, loader, AI interpreter, analytics, assistant)
■■■ data/ (sales.csv and sales.db)
■■■ static/ (index.html, style.css, script.js)
■■■ tests/ (test_analytics.py)
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

Security note: The Gemini API key is stored locally in an environment file and must never be included in a shared project archive or report. Configure the key through a local .env file and exclude that file from version control and submission packages.